

KMeans OpenMP

Mattia Baroncelli

7166468

mattia.baroncelli@edu.unifi.it

Abstract

Il progetto si concentra sull'implementazione dell'algoritmo K-Means, in particolare il confronto fra la versione sequenziale e quella parallela. K-Means è un algoritmo per l'apprendimento non supervisionato, che permette dato un insieme di punti appartenenti ad uno stesso spazio dimensionale e un valore K, di partizionare l'insieme di punti in K diversi cluster. K-Means crea i cluster assegnando ad ogni punto il centroide più vicino, andando così a minimizzare la varianza all'interno di ogni gruppo. Lo scopo principale del progetto è quindi di implementare una versione parallela dell'algoritmo tramite il framework OpenMP, valutando i vantaggi rispetto alla versione sequenziale. Si andrà inoltre a ricercare la migliore soluzione al problema della parallelizzazione.

Future Distribution Permission

The author(s) of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

L'algoritmo K-Means è un metodo che permette di formare cluster a partire da un dato insieme di punti. Questo viene fatto dando in input all'algoritmo un dataset di punti e un valore K che rappresenta il numero di cluster in cui saranno divisi i punti. L'algoritmo si basa sull'utilizzo dei centroidi, ossia K punti appartenenti al dataset, che durante l'esecuzione dell'algoritmo si andranno a spostare in base all'assegnazione dei punti del dataset ad ogni cluster. Al termine dell'algoritmo ogni punto sarà assegnato al cluster che minimizza la distanza con il proprio centroide. L'algoritmo è composto da tre fasi che vedremo nella prossima sezione ed è un algoritmo molto noto in letteratura, poichè nonostante la sua semplicità nell'implementazione risulta essere molto efficace. Nonostante questo però presenta alcune criticità, infatti all'aumentare della dimensionalità dei punti selezionati, sia per quanto riguarda il numero che per lo spazio, i tempi di esecuzione si dilatano notevolmente. Con lo scopo di superare queste limitazioni ho implementato

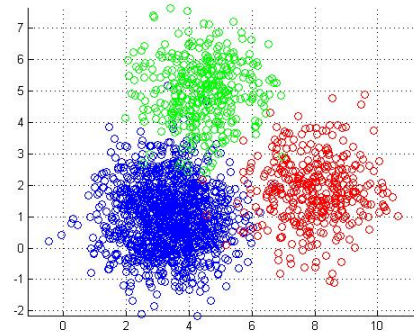


Figure 1. Esempio di clustering K-Means con evidenziate le partizioni dei punti.

la versione parallela dell'algoritmo. Attraverso il framework *OpenMP* è possibile parallelizzare parte del codice, e tramite questo andremo a valutare quanto l'impatto della parallelizzazione migliori le prestazioni dell'algoritmo.

2. Implementazione

L'algoritmo K-Means è costituito da tre fasi principali, ognuna contraddistinta da step distinti:

- Inizializzazione dei centroidi: in questa fase si andranno a selezionare casualmente K punti dal dataset che saranno utilizzati come centroidi.
- Assegnamento dei punti ai cluster: l'algoritmo andrà a cercare per ogni punto il centroide a lui più vicino e di conseguenza gli assegnerà il corrispondente cluster.
- Aggiornamento dei centroidi: una volta assegnati tutti i punti verrà fatta una media delle posizioni dei punti appartenente ad ogni cluster per aggiornare il relativo centroide.

Sull'implementazione di ogni fase ci concentreremo di seguito quando parleremo nel dettaglio del confronto fra versione sequenziale e parallela. Per quanto riguarda le condizioni di convergenza dell'algoritmo esistono varie scelte,

io ho preferito interrompere l'algoritmo quando nessun punto ha cambiato cluster dopo un'iterazione di assegnazione dei punti ai rispettivi gruppi.

2.1. OpenMP directives

Prima di iniziare con l'analisi del codice è utile vedere le direttive che andrò ad utilizzare per parallelizzare l'algoritmo nel progetto. Grazie a questo particolare tipo di istruzioni presenti in *OpenMP* è possibile creare multipli threads così da velocizzare l'esecuzione del codice, di seguito le direttive che ho usato.

- `#pragma omp parallel`: Grazie a questa istruzione è possibile generare multipli threads all'interno del programma.
- `#pragma omp for`: the *omp for* directive instructs the compiler to distribute loop iterations within the team of threads that encounters this work-sharing construct.
- `#pragma omp critical`: The *omp critical* directive identifies a section of code that must be executed by a single thread at a time.

Grazie a queste direttive non solo riusciremo a parallelizzare il codice, ma anche ad evitare problemi che potrebbero nascere proprio a causa della parallelizzazione.

2.2. Sequential version

Iniziamo l'analisi con la versione sequenziale dell'algoritmo K-Means. L'algoritmo è molto noto in letteratura e come detto prima è formato da tre parti principali. L'algoritmo è stato scritto in modo tale da potersi adattare alla versione parallela con poche aggiunte (principalmente le direttive *OpenMP*).

Listing 1. Fase di assegnamento della versione sequenziale

```
1 for (int i = 0; i < n; i++) {
2     double min_dist = std::numeric_limits<
3         double>::max();
4     int best_cluster = -1;
5
6     for (int j = 0; j < K; ++j) {
7         double dist =
8             calculateDistanceSquare(points[
9                 i].values, centroids[j]);
10        if (dist < min_dist) {
11            min_dist = dist;
12            best_cluster = j;
13        }
14    }
15    if (points[i].cluster != best_cluster) {
16        points[i].cluster = best_cluster;
17        changed = true;
18    }
19 }
```

Il primo blocco di codice che andiamo ad analizzare riguarda la fase di assegnamento di ogni punto al proprio centroide. La fase di selezione dei centroidi, infatti, non è molto interessante per l'analisi e quindi abbiamo preferito non riportarla. Nel codice si andrà ad effettuare un ciclo per ogni punto e per ogni centroide si andrà a calcolare la distanza fra quest'ultimo e il punto, andando di volta in volta ad aggiornare il parametro `min_dist` fino a trovare il centroide che minimizza la distanza con il punto selezionato. Infine se il centroide ha cambiato cluster, si va ad aggiornare il cluster e a cambiare il valore di un flag necessario per verificare la condizione di arresto. Andiamo adesso a vedere la parte del codice che riguarda il calcolo dei nuovi centroidi.

Listing 2. Fase di aggiornamento dei centroidi della versione sequenziale

```
1 std::vector<std::vector<double>>
2     new_centroids_sum(K, std::vector<double>
3         >(dimensions, 0.0));
4     std::vector<int> cluster_counts(K,
5         0);
6 for (int i = 0; i < n; ++i) {
7     int cluster_id = points[i].cluster;
8     cluster_counts[cluster_id]++;
9     for (int d = 0; d < dimensions; ++d) {
10         new_centroids_sum[cluster_id][d]
11             += points[i].values[d];
12     }
13 }
14
15 for (int j = 0; j < K; ++j) {
16     if (cluster_counts[j] > 0) {
17         for (int d = 0; d < dimensions; ++
18             d) {
19             centroids[j][d] =
20                 new_centroids_sum[j][d] /
21                 cluster_counts[j];
22         }
23     }
24 }
```

In questo blocco vengono innanzitutto inizializzati due parametri: una matrice che si occuperà di conservare le somme dei punti che appartengono ad un particolare centroide per ognuna delle sue dimensioni e un vettore che conta il numero di punti all'interno di un cluster. Si ha quindi un ciclo su ogni punto, che andrà ad incrementare il contatore del relativo cluster e andrà ad aggiornare la matrice delle somme. Una volta aggiornati questi due parametri per ogni punto si andrà ad effettuare la media dividendo i valori all'interno della matrice per il vettore delle somme di ogni centroide, ottenendo così le coordinate dei centroidi aggiornate. Senza far vedere il codice ritengo corretto citare la condizione di arresto che si verifica quando dopo un'iterazione dell'algoritmo nessun punto ha cambiato cluster rispetto all'iterazione precedente, ed è imple-

mentato con un semplice ciclo `while`.

2.3. Parallel version

Dopo aver implementato la versione sequenziale del codice vediamo adesso come è possibile passare ad una versione parallela. Le versioni parallele sviluppate in realtà sono due, che sfruttano due approcci diversi della parallelizzazione:

- Array of Structures (AoS): E' l'approccio più vicino a quanto implementato con la versione sequenziale. I dati sono organizzati in memoria in modo che il singolo punto abbia le proprie coordinate spazialmente vicine.
- Structure of Arrays (SoA): Questo approccio separa la singole componenti dei punti in array distinti, permettendo ai thread di leggere blocchi contigui di memoria durante il calcolo delle distanze.

A livello teorico quindi l'approccio SoA dovrebbe darci risultati migliori in termini di speed-up rispetto alla versione sequenziale, vedremo nella sezione degli Esperimenti se questo sarà confermato. Soffermiamoci adesso ad analizzare entrambe le versioni di codice che abbiamo implementato.

2.3.1 Array of Structures

Come detto prima questa prima versione parallela non si discosta molto da quella sequenziale, grazie all'approccio usato in precedenza è sufficiente adattare il codice prestando attenzione ad alcune problemi derivati dalla parallelizzazione. Andiamo innanzitutto a valutare la struttura della classe `Point`, che risulta essere la stessa utilizzata nella versione sequenziale

Listing 3. Implementazione dei punti nella versione AoS

```
1 struct Point {
2     int cluster;
3     std::vector<double> values;
4
5     Point(std::vector<double> v);
6 };
```

Ogni punto è quindi formato dalle proprie coordinate di appartenenza e dall'ID del proprio cluster, che è inizializzato a -1 nella fase di inizializzazione dei punti. Andiamo quindi adesso ad analizzare il codice che assegna i punti ai cluster

Listing 4. Fase di assegnamento parallela AoS

```
1 #pragma omp parallel for reduction(||: changed)
2 for (int i = 0; i < n; i++) {
3     double min_dist = std::numeric_limits<
4         double>::max();
```

```
4     int best_cluster = -1;
5
6     for (int j = 0; j < K; ++j) {
7         double dist =
8             calculateDistanceSquare(points[
9                 i].values, centroids[j]);
10        if (dist < min_dist) {
11            min_dist = dist;
12            best_cluster = j;
13        }
14    }
15    if (points[i].cluster != best_cluster) {
16        points[i].cluster = best_cluster;
17        changed = true;
18    }
19 }
```

All'atto pratico il codice risulta essere uguale alla versione sequenziale, con l'aggiunta della direttiva `#pragma omp parallel for reduction(||:changed)`, il costrutto serve a parallelizzare il ciclo `for` sottostante che effettua il calcolo delle distanze fra punti e centroidi e assegna ogni punto a un centroide, in questo caso le variabili `min_dist` e `best_cluster` venendo dichiarate all'interno del ciclo diventano private per il thread che sta eseguendo. Lo stesso non si può dire del flag `changed` che viene inizializzato fuori dal ciclo e che può portare ad un comportamento indefinito. Se due thread cercano di scrivere nello stesso momento in quella cella di memoria possono verificarsi rallentamenti o nella peggiore delle ipotesi la corruzione del dato e di conseguenza un funzionamento errato dell'algoritmo (potrebbe fermarsi prima). Per risolvere questo problema utilizziamo il costrutto `reduction`. OpenMP andrà a creare delle copie locali della variabile `changed`. Una volta terminato il ciclo OpenMP fonde le variabili locali tramite l'operatore `OR` (che gli abbiamo passato). Il risultato pertanto sarà lo stesso ma evitando il problema citato in precedenza.

Listing 5. Fase di aggiornamento dei centroidi parallela AoS

```
1 #pragma omp parallel
2 {
3     std::vector<std::vector<double>>
4         local_centroids_sum(K, std::vector<
5             double>(dimensions, 0.0));
6     std::vector<int> local_cluster_counts(K, 0)
7     ;
8
9     #pragma omp for
10    for (int i = 0; i < n; ++i) {
11        int cluster_id = points[i].cluster;
12        local_cluster_counts[cluster_id]++;
13        for (int d = 0; d < dimensions; ++d)
14        {
15            local_centroids_sum[cluster_id][d]
16                += points[i].values[d];
17        }
18    }
19 }
```

```

12     }
13 }
14
15 #pragma omp critical
16 {
17     for (int j = 0; j < K; ++j) {
18         cluster_counts[j] +=
19             local_cluster_counts[j];
20         for (int d = 0; d < dimensions; ++
21             d) {
22             new_centroids_sum[j][d] +=
23                 local_centroids_sum[j][d];
24         }
25     }
26
27     for (int j = 0; j < K; ++j) {
28         if (cluster_counts[j] > 0) {
29             for (int d = 0; d < dimensions; ++d)
30                 {
31                     centroids[j][d] =
32                         new_centroids_sum[j][d] /
33                         cluster_counts[j];
34                 }
35     }
36 }

```

La fase di aggiornamento dei centroidi si apre con, oltre alla dichiarazione degli accumulatori di somme e numero di punti, anche con quella di accumulatori locali per ogni cluster, che assegneremo ad ogni thread. Anche stavolta tramite la direttiva `#pragma omp for`, andremo a parallelizzare il ciclo. In maniera analoga a prima ogni thread andrà ad aggiornare i propri accumulatori. Una volta terminato l'aggiornamento degli accumulatori per ogni punto, è necessario aggiornare gli accumulatori globali, andandogli ad assegnare la somma dei rispettivi contatori parziali. Questa operazione può portare al verificarsi di una race condition, dove due threads leggono un valore e provano a sovrascrivere gli accumulatori globali con i propri valori, rischiando di portare a risultati errati. Per risolvere questo problema abbiamo adottato il costrutto `#pragma omp critical` che assicura che un solo thread alla volta aggiorni i contatori globali.

2.3.2 Structure of Arrays

Andiamo a vedere in questa sezione come è possibile implementare l'algoritmo con il paradigma SoA. Di seguito il codice che illustra la struttura l'implementazione dei punti con questa tecnica.

Listing 6. struct Implementazione dei punti nella versione SoA

```

1 struct DatasetSoA {
2     int num_points;

```

```

3     int dimensions;
4
5     std::vector<double> coords;
6     std::vector<int> clusters;
7 };

```

Il dataset viene implementato tramite la struct `DatasetSoA` che ha come attributi, il numero totale di punti e di dimensioni, il vettore che rappresenta le coordinate dei punti e un array separato per etichette dei clusters. La funzione `convertToSoA` si occuperà di convertire i punti passati nel formato AoS, in questo formato, tramite un ciclo `for`.

Listing 7. ciclo `for` all'interno di `convertToSoA`

```

1 for (int i = 0; i < soa.num_points; ++i) {
2     soa.clusters[i] = aos_dataset[i].cluster
3     ;
4     for (int d = 0; d < soa.dimensions; ++d)
5     {
6         // Indice piatto: (riga *
7         // numero_colonne) + colonna
8         soa.coords[i * soa.dimensions + d] =
9             aos_dataset[i].values[d];
10    }
11 }

```

In particolare la funzione prende in input il dataset in formato AoS e innanzitutto assegna il medesimo cluster di ogni punto alla rappresentazione in formato SoA, successivamente attraverso la tecnica del flat index, costruisce l'array di coordinate. In questo modo è possibile passare facilmente dalla rappresentazione AoS a SoA. Anche l'algoritmo avrà necessità di essere adattato, in particolare, oltre a una diversa versione dell'algoritmo per calcolare la distanza, che sfrutta anch'esso il flat index; abbiamo una differenza sostanziale nella terza fase dell'algoritmo.

Listing 8. Fase di aggiornamento dei centroidi SoA

```

1 std::vector<double> new_centroids_sum(K *
2     dims, 0.0);
3 std::vector<int> cluster_counts(K, 0);
4
5 #pragma omp parallel
6 {
7     std::vector<double> local_centroids_sum(K *
8         dims, 0.0);
9     std::vector<int> local_cluster_counts(K, 0)
10    ;
11
12    for (int i = 0; i < n; ++i) {
13        int cluster_id = dataset.clusters[i];
14        local_cluster_counts[cluster_id]++;
15
16        for (int d = 0; d < dims; ++d) {
17            local_centroids_sum[cluster_id * dims
18                + d] += dataset.coords[i * dims +
19                d];
20        }
21    }
22 }

```

```

16 }
17 {
18 {
19     for (int j = 0; j < K; ++j) {
20         cluster_counts[j] +=
21             local_cluster_counts[j];
22         for (int d = 0; d < dims; ++d) {
23             new_centroids_sum[j * dims + d] +=
24                 local_centroids_sum[j * dims +
25                     d];
26         }
27     }
28 }
29 }
30
31 for (int j = 0; j < K; ++j) {
32     if (cluster_counts[j] > 0) {
33         for (int d = 0; d < dims; ++d) {
34             centroids[j * dims + d] =
35                 new_centroids_sum[j * dims + d] /
36                 cluster_counts[j];
37         }
38     }
39 }

```

Nelle operazioni compiute il codice è molto simile a quello scritto per la versione AoS. Fin da subito però è possibile notare come sia gli accumulatori globali che quelli locali siano cambiate. In questa versione infatti si richiede per le somme dei centroidi una allocazione per un blocco continuo, al contrario delle metriche allocate in precedenza. Inoltre il costrutto di parallelizzazione è diventato `#pragma omp for schedule(static)`, che forza l'assegnazione dei blocchi contigui ad uno stesso thread (cosa che è stata aggiunta anche alla fase di assegnamento dei punti). Infine sia il calcolo della somma dei centroidi e della media finale vengono fatte utilizzando la rappresentazione piatta.

3. Esperimenti

Iniziamo adesso la sezione del report che riguarda gli esperimenti effettuati, abbiamo visto come i tre diversi tipi di implementazione variano a livello di codice, adesso vediamo quanto variano a livello di prestazioni. Per farlo è necessario partire innanzitutto dalla creazione del dataset.

3.1. Dataset

I dataset sono stati creati attraverso una funzione Python creata appositamente `generate_dataset`. La funzione prende in input:

- Numero di punti (N)
- Numero di cluster (K)
- Dimensione del piano su cui giacciono i punti

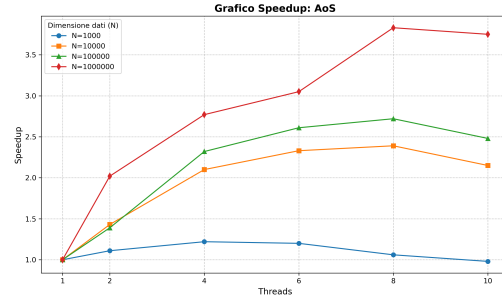


Figure 2. Speedup della versione AoS all'aumentare del numero di punti.

- Parametro di dispersione dei punti

e restituisce in output un dataset in formato csv, che contiene sulle righe le coordinate di ogni punto. Poiché k-means è un algoritmo non supervisionato nel dataset non è presente il cluster di appartenenza, nonostante ciò il numero di cluster è un parametro in input per creare dataset che abbiamo punti che vadano a formare cluster definiti. Per convertire i punti in formato SoA verrà poi usata la funzione apposita definita nella sezione precedente; la conversione come era logico aspettarsi non concorrerà alla misurazione del tempo di esecuzione dell'algoritmo.

3.2. Array of Structures

Iniziamo quindi gli esperimenti valutando le prestazioni della tecnica Structure of Arrays su vari dataset. Il primo esperimento verrà fatto aumentando la dimensione del numero di punti (N), mantenendo fermo il numero di cluster K a 10. Andremo inoltre ad aumentare gradualmente il numero threads in esecuzione, per verificare se c'è un miglioramento delle performance all'aumentare dei threads. Tutti gli esperimenti sono stati fatti estraendo la mediana dei risultati su un numero variabile di esecuzione.

Nel grafico in Figura 2 è possibile notare come sull'asse Y sia presente il parametro *Speedup* che è calcolato dividendo il tempo di esecuzione della versione sequenziale per quello della versione parallela. Si può osservare dal grafico blu (quello riguardante N=1000) come in questo caso la parallelizzazione non porta vantaggi degni di nota facendo ottenere uno speedup massimo di 1.20x. Aumentando la dimensione del dataset però la parallelizzazione porta dei vantaggi rilevanti. In particolare quando il numero di punti nel dataset è pari a 1 milione, già solo la parallelizzazione fra due threads porta a dimezzare i tempi di esecuzione rispetto alla versione sequenziale. Proprio in questo caso è possibile notare il miglioramento più evidente; con 8 cores le prestazioni sono quasi quadruplicate, raggiungendo uno speedup di 3.85x. E' interessante notare come all'aumentare del numero di punti da classificare la parallelizzazione porti a risultati sempre migliori.

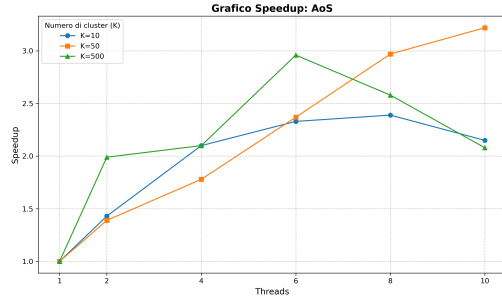


Figure 3. Speedup della versione AoS all'aumentare del numero di cluster.

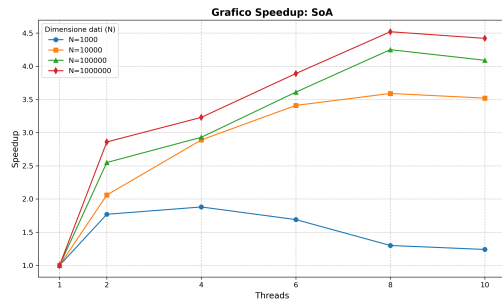


Figure 4. Speedup della versione SoA all'aumentare del numero di punti.

Con il grafico in Figura 3, andiamo a valutare le prestazioni delle due versioni variando stavolta il numero di cluster e tenendo fisso il numero di punti a 20000. In questo caso abbiamo comportamenti diversi per i diversi valori di K. Per K=50 infatti abbiamo un continuo incremento delle prestazioni all'aumentare del numero di threads. Quando il numero di cluster viene aumentato a 500, il picco massimo delle prestazioni si ottiene con 6 threads e uno speedup di 2.96x. Aumentare il numero di threads porta ad una degradazione delle performance. Il grafico blu mostra un andamento analogo all'esperimento precedente, con uno speedup massimo di 2.39x.

3.3. Structure of Arrays

In questa sezione ripeteremo gli esperimenti fatti per la versione AoS, stavolta usando il paradigma Structure of Arrays.

In Figura 4 possiamo vedere come le performance raggiunte con questa versione sono migliori di quelle con AoS. In particolare a partire con i dataset di dimensione più grande si ottiene uno speedup di 4.52x, che dimostra come questo approccio risulti vincente quando si hanno grandi quantità di dati.

Anche aumentando i cluster è possibile notare come l'approccio SoA permetta di migliorare le performance dell'algoritmo molto di più rispetto alla controparte AoS.

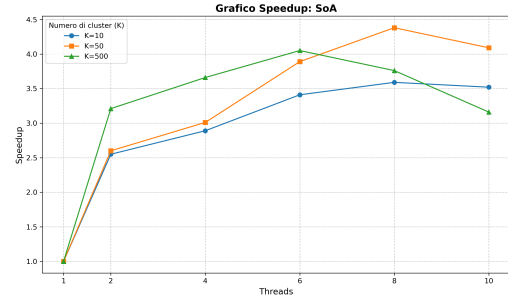


Figure 5. Speedup della versione SoA all'aumentare del numero di cluster.

Si riscontra però un comportamento analogo a prima; con K=500 si ottengono prestazioni peggiori rispetto a K=50. Questo probabilmente è dovuto dalla fase di aggiornamento dei centroidi dove i vari threads devono comunicare per effettuare la media. Inoltre la sezione critica è rappresentata da un ciclo sui centroidi e pertanto al crescere di K si verificheranno più di frequente situazioni di attesa.

3.4. Array of Structures vs Structure of Arrays

Dopo aver analizzato i due approcci indipendentemente in questa sezione andremo a confrontare i risultati numerici di entrambe le versioni, per misurare la differenza in termini di performance dei due metodi.

Table 1. Confronto delle prestazioni fra AoS e SoA (N=100000)

Threads	AoS	SoA	Efficienza SoA vs AoS
2	1.39x	2.06x	1.48x
4	2.32x	2.93x	1.26x
6	2.61x	3.61x	1.38x
8	2.72x	4.25x	1.56x
10	2.48x	4.09x	1.65x

La Tabella 1 è estratta dal caso dove il numero di elementi è uguale a 100000, ma rispecchia tutti i casi che abbiamo analizzato. Sono illustrate le prestazioni dell'algoritmo in termini di speedup sia per la versione AoS che per SoA. Come è possibile notare l'approccio Structure of Arrays risulta essere sempre migliore di Array of Structures, confermando i risultati visti nella teoria. In particolare possiamo vedere come l'efficienza di questo metodo (calcolata dividendo lo speedup ottenuto con SoA per quello ottenuto con AoS), cresca con l'aumentare dei threads usati della parallelizzazione. Pertanto nel caso di quantità di dati maggiori e quindi un maggior numero di thread è sempre da preferire l'approccio Structure of Arrays.

La Tabella 2 mostra i risultati in modo analogo a prima, ma stavolta prendendo un valore elevato del numero di cluster, invece del numero di punti. Anche in questo scenario le performance ottenute tramite SoA sono migliori. E' possi-

Table 2. Confronto delle prestazioni fra AoS e SoA (K=50)

Threads	AoS	SoA	Efficienza SoA vs AoS
2	1.39x	2.60x	1.87x
4	1.78x	3.01x	1.69x
6	2.37x	3.89x	1.64x
8	2.97x	4.38x	1.47x
10	3.22x	4.09x	1.27x

bile notare però un pattern opposto a prima, stavolta infatti all'aumentare del numero di threads la scelta dell'approccio risulta meno importante. Nonostante ciò con il numero di threads ottimale (ossia 8), l'approccio Structure of Arrays risulta migliore di 1.47 punti.

4. Conclusioni

Il progetto ha quindi analizzato l'efficienza dell'algoritmo K-Means, facendo un confronto fra versione sequenziale e le due principali strategie di parallelizzazione, utilizzando OpenMP. Alla luce degli esperimenti condotti quindi, emergono le seguenti conclusioni:

- L'algoritmo ha dimostrato di essere un ottimo candidato per la parallelizzazione. Entrambi gli approcci (AoS e SoA) si sono dimostrati particolarmente efficaci.
- All'aumentare del numero di elementi del dataset la parallelizzazione mostra il suo lato migliore. All'aumentare del numero di threads si ottengono risultati di gran lunga migliori rispetto alla versione sequenziale.
- Aumentando il numero di cluster la parallelizzazione offre ancora un gran risultato, superando sempre le performance della versione sequenziale. Per un valore di K alto il numero ottimale di threads risulta essere più basso. Rendendo deleterio fornire ulteriori core.
- Il confronto fra Array of Structures e Structures of Arrays ci ha permesso di dimostrare che la seconda versione è quella che offre le prestazioni migliori, sia con una grande dimensione del dataset che con un alto numero di cluster.